

PATIENCE: A CARD GAME

Aidan Flett

AIF7@ABER.AC.UK 02/05/2025

Contents

Introduction	2
Design.....	3
Class diagram:	3
Textual description of classes	4
Card	4
Cardpile	4
Deck.....	4
Table.....	4
Menu	5
Pseudo-code	6
Testing.....	7
Test table	7
Testing report.....	9
Evaluation	10
What difficulties I faced	10
What remains to be done	10
What did I learn	10
What marks I should be awarded and why	10
References	11

The word count excluding references and first two pages is 2098.

Introduction

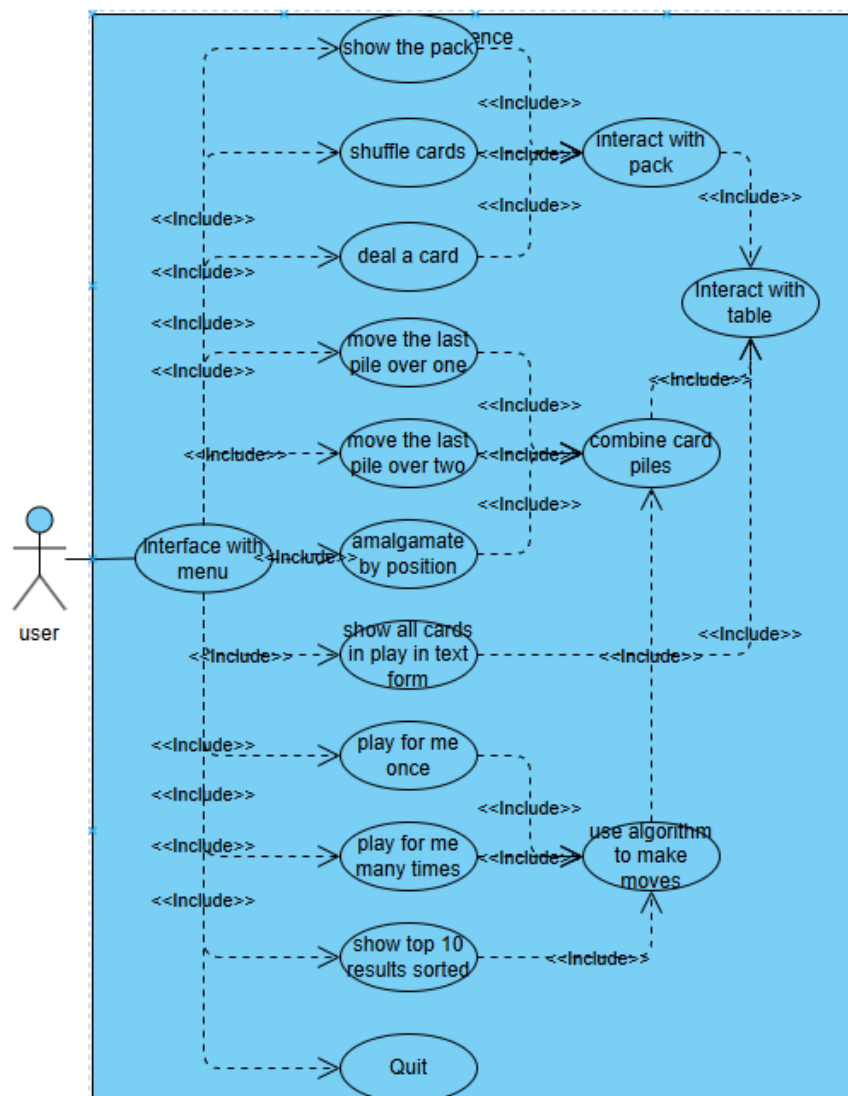
Within this report I will explain various aspects of my project.

Firstly, I will explain its design involving showing a class diagram, a textual description of each class within my algorithm with a rationale behind it and an explanation of its relationships, and a pseudocode example of the most complicated algorithm within it.

Next, I will test the project by utilising a test table including descriptions of each test, the result, and screenshots as appropriate. I will then discuss the test data I used.

Lastly, I will evaluate the project. This will involve explaining how I went about completing the assignment, what I found difficult, what I didn't implement, what I learnt, as well as what mark I am expecting to receive and why.

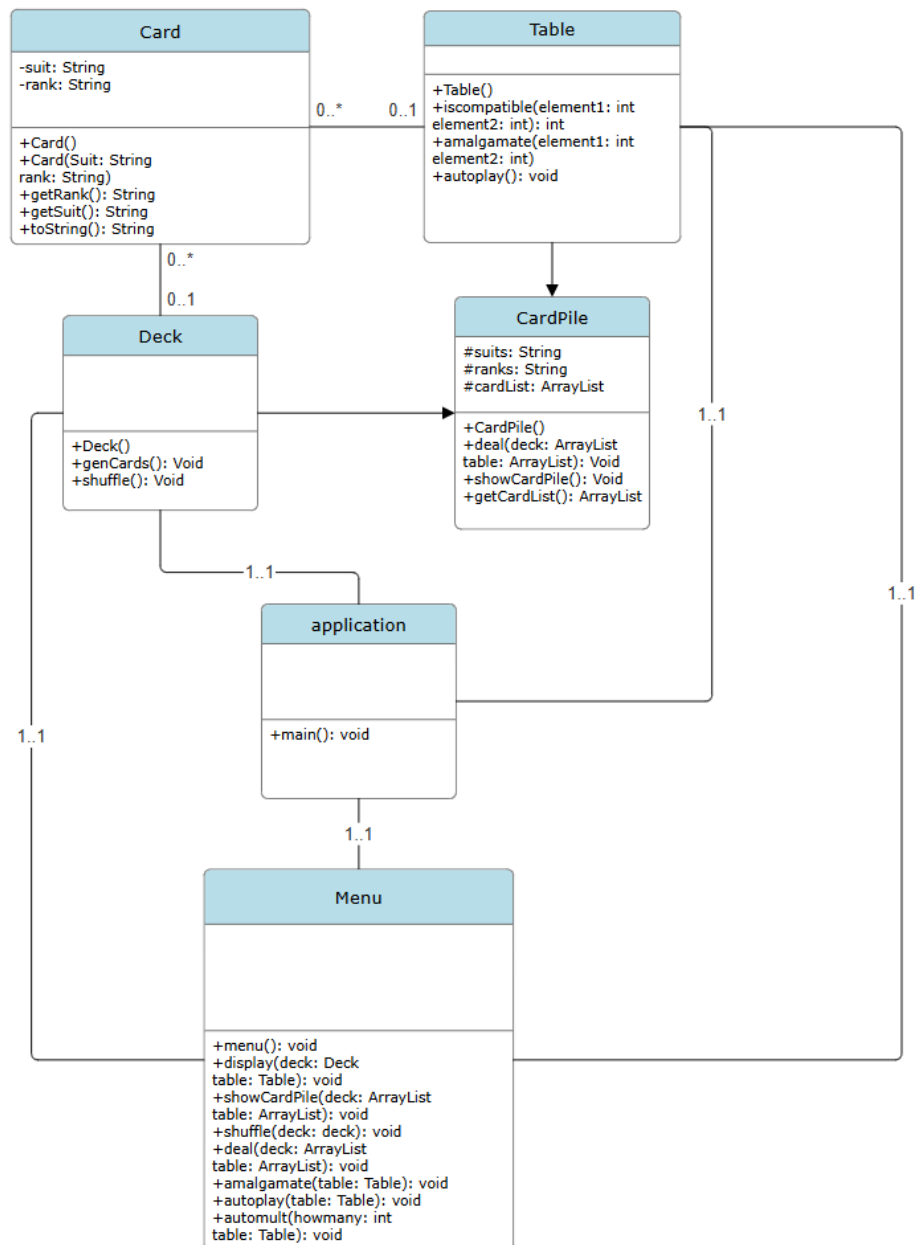
Below is a use case diagram of functions the user will need to utilise and their requirements:



Design

Class diagram:

Below is a class diagram explaining the structure of the classes, and their relationships in the project:



Textual description of classes

In this section of my report, I will be explaining and justifying the classes I implemented:

Card

My card class is (as the name implies) used to create and store objects to represent each card within a deck of playing cards.

This was necessary to implement as they form the basis of any card game, and serve as a useful building block for the rest of the project. They have two attributes being suit and rank, I stored both of these as strings as it made it easier to concatenate them for their toString method.

As well as the previously mentioned toString method which concatenates and returns the two attributes as strings, there is a constructor which passes in two strings to set the suit and rank, as well as getter methods for both the suit and rank. These allow me to carry out comparisons with other objects of the same class to see if either attribute is the same. This is the basis of the amalgamation mechanic.

Cardpile

Cardpile is a class that can be used to store any ArrayList of cards. This is primarily used as the super class for both Deck and Table. I utilised inheritance in order to implement encapsulation, this is because although the deck and table would need a lot of the same functions and attributes, they still have a few mutually exclusive functionalities which the other should not be given access to.

The first two attributes suits and ranks are strings which are used to store the possible suits and ranks of cards. They are utilised within Deck by having a method iterate through them to create all possible combinations of cards. The other attribute is an ArrayList called cardList, which to store collections of cards being utilised in Deck and Table.

The first method I implemented was showCardPile. This is used to iterate through a collection of cards concatenating their values, printing the combined string of them at the end. One of the other methods was getCardList. This was a getter method for the ArrayList cardList allowing the other classes to interact with it. There is also a deal method which accesses Cardpiles subclasses, adding the front of the deck to the back of the table, before removing the card it copied from the deck. There is also a default constructor method.

Deck

Deck is a subclass of Cardpile which specifically represents the cards which are yet to be dealt. It doesn't contain any attributes, however it has several methods.

The first of these is a basic constructor. There is also a function called genCards which uses a nested for loop to iterate through every combination of suit and card and creates a card object for it. The final function in the class is shuffle, this uses the command shuffle from the collections library and uses it to randomise the order of objects in the cardList.

Table

Table is another subclass of Cardpile which is used to interact with all of the cards currently in play or "on the table" thus the name. It also doesn't have any specific attributes but contains a significant amount of methods.

One of these is the amalgamate method. This is used to combine two different elements of the cardList associated with the class taking in two integers to represent the positions of which card is replacing which. It sets the value of the second element and sets it to be equal to the first, before

removing the first element from the ArrayList. Before doing this it uses the second method: `isCompatible`. This checks if the two positions given are the correct distance apart and if the cards they reference have the same suit or rank. If any of these tests fail it returns 0, and if they all succeed it returns the distance between the two elements. This is done so that the next method can work. The method in question is `autoPlay`. This iterates through the `cardList` looking for possible moves. First it does an initial sweep looking for the furthest moves possible, playing the first it finds. However, if this fails it looks for any moves playing one if it finds one and then ending the method whether or not one is found.

Menu

Menu is an abstract class I used to display a menu and receive the users input. This is responsible for calling almost every method of other classes. As a result of this, it has a ridiculous amount of methods.

The first of these is `menu`, this prints out each command the user can give. The next method `display` then calls the menu and prompts the user to enter one of the commands, it then passes the deck, table, or any other data necessary into other functions, prompting the user for other data as required. One of the functions it can call is `showCardPile`, this passes either the deck or table into the `cardPile` classes' `showCardPile` function. The same applies for the methods: `shuffle`, `deal`, `amalgamate`, and `autoplay`, each of which feedback to the functions sharing their names which were mentioned earlier. The only other function is `automult`, which calls `autoplay` a specified number of times.

Pseudo-code

For this section I have been asked to create pseudo-code of the section of the code I believe to be the most complex. For this I have chosen the autoplay method from the Table class. This is because it involves a nested for loop, as well as several considerations which had to be made to ensure I didn't produce an error when amalgamating the cards. The pseudo-code can be seen below:

```
boolean threeAcross = false;
int ival = -1;
int jval = -1;
for i in table.cardlist:
    for j in table.cardlist:
        if iscompatible(i, j) == 3:
            threeAcross = true;
            ival = i;
            jval = j;
        endif;
    endfor;
endfor;
if threeAcross == false:
    for i in table.cardlist:
        for j in table.cardlist:
            if iscompatible(i, j) == 1:
                ival = i;
                jval = j;
            endif;
        endfor;
    endfor;
    if ival > -1 && jval > -1:
        amalgamate(table, ival, jval);
    endif;
```

Testing

In this section of the report I will be testing my code to see if it meets the functional requirements listed.

Test table

ID	Requirement	Description	Inputs	Expected outputs	Pass/fail	comments
1	Fr1	Show the pack	Entering the associated command into the command line interface	Prints out each card in the pack	Pass	Prints out the pack without any alterations if ran before other commands (screenshot 1), also prints out the pack after shuffling (screenshot 2) or dealing a card (screenshot 3)
2	Fr2	Shuffling the pack	Showing the pack in the CLI, entering the command to shuffle into the CLI, then printing the pack again.	The second print statement will have the cards in a randomized order	Pass	Prints the pack in different orders the first and second time (screenshot 2)
3	Fr3	Dealing a card	Printing the deck, pressing the button in the CLI that corresponds to dealing a card, then printing the pack again.	The dealt card will appear in the print statement within the deal function that shows the table, and said card will no longer appear in print statements of the pack.	Pass	Happens as described (screenshot 3)
4	Fr4	Move the last pile one over	Entering the command when the two cards are compatible	First card is replaced by the second, second card is removed	Pass?	Does as described, but shows an error (screenshot 4)
			Entering the command when the	Nothing changes	Pass?	Does as described, but shows wrong

			two cards are incompatible			error (screenshot 5)
			Entering the command when there is one card	Nothing changes	Fail	Causes an out of bounds error (screenshot 6)
5	Fr5	Move the last pile two over	Entering the command when the two cards are compatible	Combines the two cards	pass	Does as described (screenshot 7)
			Entering the command when the two cards are incompatible	Does nothing	pass	Does as is described (screenshot 8)
6	Fr6	Combine two specified cards	Select two compatible cards	Combines cards	pass	Combines as described (screenshot 9)
			Select two incompatible cards	Does nothing	pass	Does nothing (screenshot 10)
7	Fr7	Show all cards in play	Deal 5 times and then show the table	Prints the 5 dealt cards	pass	Does as described (screenshot 11)
			Show table without dealing	Prints a blank line	pass	Does as described (screenshot 12)
8	Fr8	Make an automatic move once	Attempt when there is a valid move	Combines two cards	pass	Does as described (screenshot 13)
			Attempt when there is not a valid move	Does nothing	pass	Does as described (screenshot 14)
9	Fr9	Make an automatic move multiple times	Attempt an amount of move $\leq$ the amount of possible moves	Does the specified amount of moves, prioritizing the furthest moves	Pass?	Does as described but doesn't prioritize furthest moves (screenshot 15)
			Attempt an amount of moves $>$ the amount of possible moved	Does the maximum amount of moves	pass	Does as described (screenshot 16)
			Attempt 0 moves	Does nothing	pass	Does as described (screenshot 17)

10	Fr10	Show the top 10 scores	Entering the associated button in the CLI	Prints top 10 scores, sorted	Fail	Not implemented, button does nothing
11	Fr11	Quit	Pressing the associated button on the CLI	Exit code 0 shows on the CLI	pass	Does as described (screenshot 18)

Testing report

The tests were mostly successful, however there were still multiple occasions where unintentional things happened, such as the out of bounds error and false “no cards on table” error message in test 4. This indicated that I would need to do further examination of the edge cases in my methods to hammer out bugs.

Evaluation

What difficulties I faced

The most significant challenge I tackled during the project was the creation of my menu class. As I tested it, it became clear that many of the methods in other classes only functioned when called within the class. Because of this I had to expend a large amount of time changing the way they referenced objects within the class to still function when the object/attribute it's operating on is passed in from outside of the class. An example of this would be me attempting to figure out why my amalgamation method wasn't working before realising all of the calls within it were using `this.table` rather than the `table` object I was passing in.

What remains to be done

I was unable to implement the scoring system or the graphical interface. This was partly due to difficulty and partly due to my own poor time management. I feel as though if I started the project earlier and was more well rested at certain points during its creation, I would've succeeded in implementing these.

What did I learn

Throughout this project I became a lot more familiar with access modifiers such as `static` and `protected` and their practical uses. Examples of this include making the methods in `Menu` `static` and the attributes in `CardPile` `protected`.

What marks I should be awarded and why

I am expecting to receive: I am expecting to receive 11 marks on the documentation and design due to my lacklustre diagrams and textual description, 18 marks on the functional requirements due to all but one working, 2 marks on non functional requirements due to only one working, 8 marks on implementation and design due to several inefficient choices and unfamiliarity with aspects such as `jdoc`, 4 marks on inheritance, 7 on testing, 4 on the evaluation, and 0 on creativity and innovation. This would make for a total of 54 marks.

References

Screenshot 1:

As seen I started the program and immediately ran the command to show the deck

```
"C:\Program Files\jdk-23.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.2.2\lib\idea_rt.jar=60847:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.2.2\bin" -jar C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.2.2\bin\idea_rt.jar 60847
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
1
AS 2S 3S 4S 5S 6S 7S 8S 9S TS JS QS KS AC 2C 3C 4C 5C 6C 7C 8C 9C TC JC QC KC AD 2D 3D 4D 5D 6D 7D 8D 9D TD JD QD KD AH 2H 3H 4H 5H 6H 7H 8H 9H TH JH QH KH
```

Screenshot 2:

As seen, I printed the deck showing all cards in their initial order, then shuffled and printed again revealing them to be in a different order:

```
1
AS 2S 3S 4S 5S 6S 7S 8S 9S TS JS QS KS AC 2C 3C 4C 5C 6C 7C 8C 9C TC JC QC KC AD 2D 3D 4D 5D 6D 7D 8D 9D TD JD QD KD AH 2H 3H 4H 5H 6H 7H 8H 9H TH JH QH KH
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
2
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
1
JH TS JD 7S 9S AS 6H 7H 4S KS BS 3H TD 4C 5S JS 7D QD TC 6S KD 9C 2D 3D 2H 3S 5D 8D 4H JC 3C 2S 5H 9D 8C AH QC 4D QS 5C 9H TH 8H AD 6C AC KC 2C 6D QH KH 7C
```

Screenshot 3:

The screenshot shows the I printed the deck, dealt a card, the dealt card appears in the table due to the print statement embedded in the deal card functionality, and then the deal card no longer appears in the printed deck.

```
1
AS 2S 3S 4S 5S 6S 7S 8S 9S TS JS QS KS AC 2C 3C 4C 5C 6C 7C 8C 9C TC JC QC KC AD 2D 3D 4D 5D 6D 7D 8D 9D TD JD QD KD AH 2H 3H 4H 5H 6H 7H 8H 9H TH JH QH KH
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
3
AS
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
1
2S 3S 4S 5S 6S 7S 8S 9S TS JS QS KS AC 2C 3C 4C 5C 6C 7C 8C 9C TC JC QC KC AD 2D 3D 4D 5D 6D 7D 8D 9D TD JD QD KD AH 2H 3H 4H 5H 6H 7H 8H 9H TH JH QH KH
```

Screenshot 4:

The screenshot shows me successfully combining two cards, but getting an error message while doing it

```
AS 2S
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
4
Invalid, no cards on the table
2S
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
```

Screenshot 5:

The screenshot shows two incompatible cards failing to combine, but giving the wrong error message

```
2S AD
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
4
Invalid, no cards on the table
2S AD
```

Screenshot 6:

The screenshot shows of me attempting to amalgamate two cards when only one is on the table, resulting in an out of bounds error

```
7
AS
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
4
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index -1 out of bounds for length 1 <3 internal lines>
    at java.base/java.util.Objects.checkIndex(Objects.java:365)
    at java.base/java.util.ArrayList.get(ArrayList.java:428)
    at Table.iscompatible(Table.java:22)
    at Table.amalgamate(Table.java:43)
    at Menu.amalgamate(Menu.java:96)
    at Menu.display(Menu.java:48)
    at application.main(application.java:17)
```

Screenshot 7:

Two cards successfully combine

```
AS 2S 3S 4S
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
5
4S 2S 3S
```

Screenshot 8:

Attempts to combine two incompatible cards, leading to nothing changing

```
4S 2S 3S 5S QC
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
5
4S 2S 3S 5S QC
```

Screenshot 9:

Combines the two selected cards

```
4S 2S 3S 5S QC
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
6
how many cards from the left is the card you would like to move, if it is the leftmost enter 0
0
how many cards from the left is the card you would like to move it to, if it is the leftmost enter 0
1
2S 3S 5S QC
```

Screenshot 10:

Does nothing, as is described

```
2S 3S 5S QC
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
6
how many cards from the left is the card you would like to move, if it is the leftmost enter 0
0
how many cards from the left is the card you would like to move it to, if it is the leftmost enter 0
3
2S 3S 5S QC
```

Screenshot 11:

Successfully prints all 5 cards

```
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
7
AS 2S 3S 4S 5S
```

Screenshot 12:

Prints a blank line, as intended

```
"C:\Program Files\jdk-23.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2023.3\lib\idea_rt.jar" 7621
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
7
options:
```


Screenshot 13:
Combines cards

```
AS 2S
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
8
2S
```

Screenshot 14:
Does nothing as intended

```
2S
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
8
2S
```

Screenshot 15:
Makes 2 moves, as specified. Although, it doesn't prioritize the furthest moves

```
2S 3S 4S 5S 6S 7S 8S 9S
options:
enter 1 to show the deck
enter 2 to shuffle the deck
enter 3 to deal a card
enter 4 to move the last card onto the previous card
enter 5 move the last card onto the card 2 before it
enter 6 to combine two specified cards
enter 7 to show all cards in play
enter 8 to make an automatic move once
enter 9 to make an automatic move a specified number of times
enter 10 to show the top 10 scores
enter 11 to quit
9
how many times would you like to play?
2
2S 3S 4S 5S 9S 7S 8S
2S 3S 4S 8S 9S 7S
```

